

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

2.1 Actividades y recursos app

Carlos Vázquez Regueiro
carlos.vazquez.regueiro@udc.es

Índice

1. **Actividad**
2. Ciclo de vida de una actividad
3. Estado de una actividad
4. Gestión básica de eventos en layouts
5. Recursos de una app
6. Acceso a los recursos
7. Recursos alternativos

Actividad

- Una actividad es un componente clave de una app:
  - Es la encargada de mostrar y gestionar una “pantalla”
  - Simplificando: cada pantalla estará gestionada por una única actividad
- Una actividad debe:
  - **Configurar su interfaz gráfica**: estática (XML) o dinámica (Java,...)
  - **Gestionar todos los eventos** del usuario con dicha interfaz
- Puede ser invocada por varias vías (en la misma app o desde otras apps)
  - Por tanto: minimizar las dependencias entre actividades (componentes)

Actividad

- Una actividad puede ser también el punto de inicio de una app
  - Debe estar indicado en el manifiesto
  - También se pueden indicar otros atributos, permisos, etc
- Puede recibir parámetros en su inicio o no (se debe gestionar este aspecto)
- En las versiones recientes de Android se pueden configurar en:
  - Multi-ventanas (se muestran dos o más actividades simultáneamente)
  - Multi-pantallas (pantallas secundarias)

Actividad

- Una actividad se crea **heredando** de una clase Activity o AppCompatActivity
- Es una clase abstracta: es obligatorio **sobrecargar el método onCreate()**
  - En dicho método **se crea la interfaz: SetContentView(*layout*)**
  - Obvio: cada actividad posee su propia “pantalla”
  - Se puede construir (o adaptar) dinámicamente por programación
- También se debe incluir todo el código para gestionar las interacciones del usuario con dicha pantalla (botones, edición de texto, etc.)

Índice

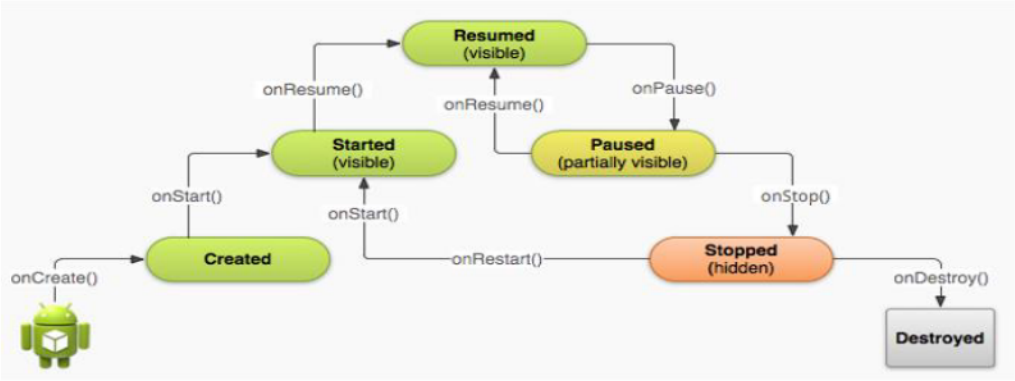
1. Actividad
2. **Ciclo de vida de una actividad**
3. Estado de una actividad
4. Gestión básica de eventos en layouts
5. Recursos de una app
6. Acceso a los recursos
7. Recursos alternativos

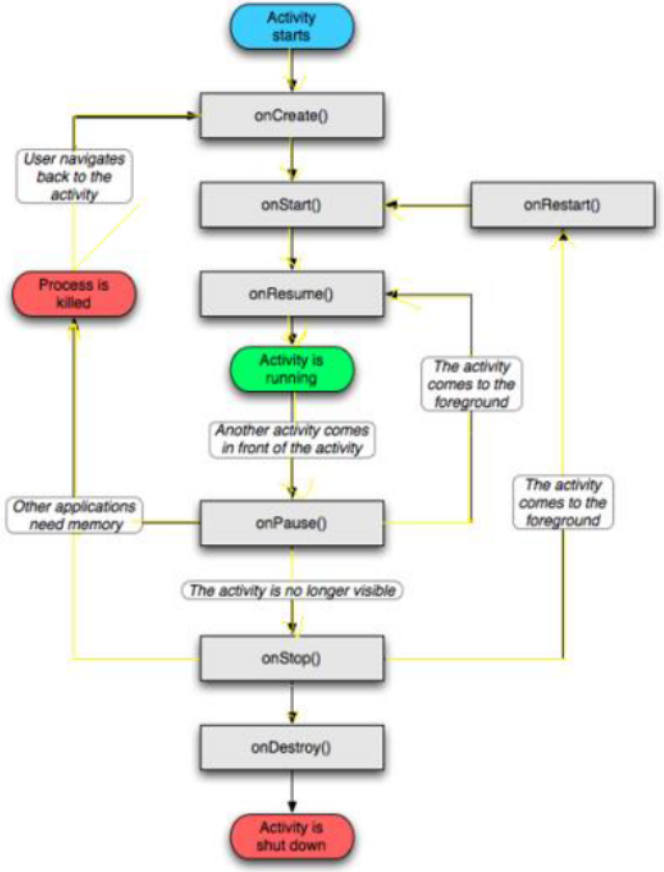
Ciclo de vida de una actividad

- Cualquier componente **puede ser interrumpido (reanudado) en cualquier momento**
- Una actividad se ejecuta durante más tiempo: su ciclo de vida es el más complejo
- El sistema operativo **(Android) gestiona cada paso** y lanza cada evento:
  - Se puede “capturar” un evento sobrecargando el método apropiado
  - Se puede añadir código adicional para gestionar la propia app

Ciclo de vida de una actividad

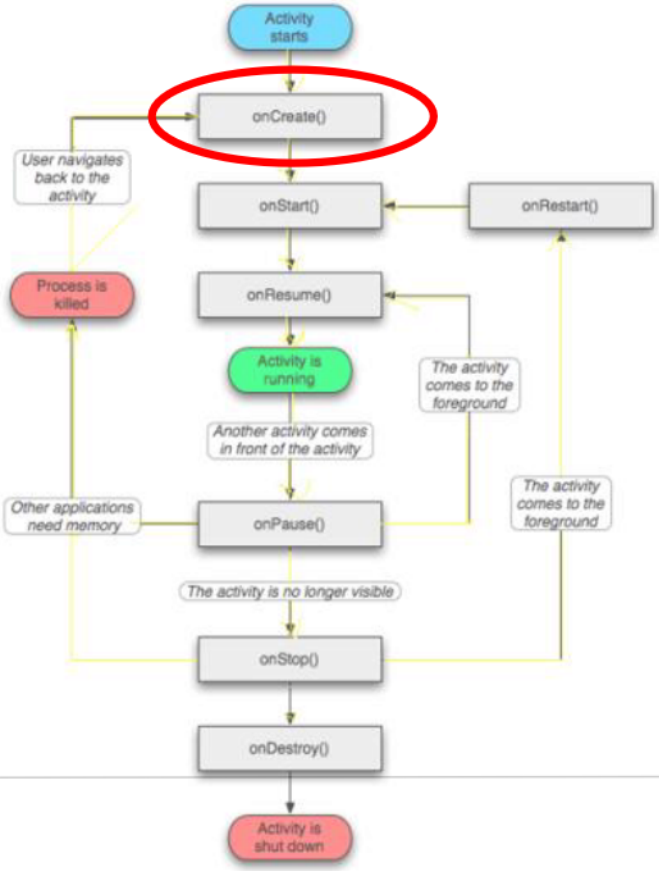
- Esquema básico con 2 representaciones (grafos):
  - Centrado en los **estados**





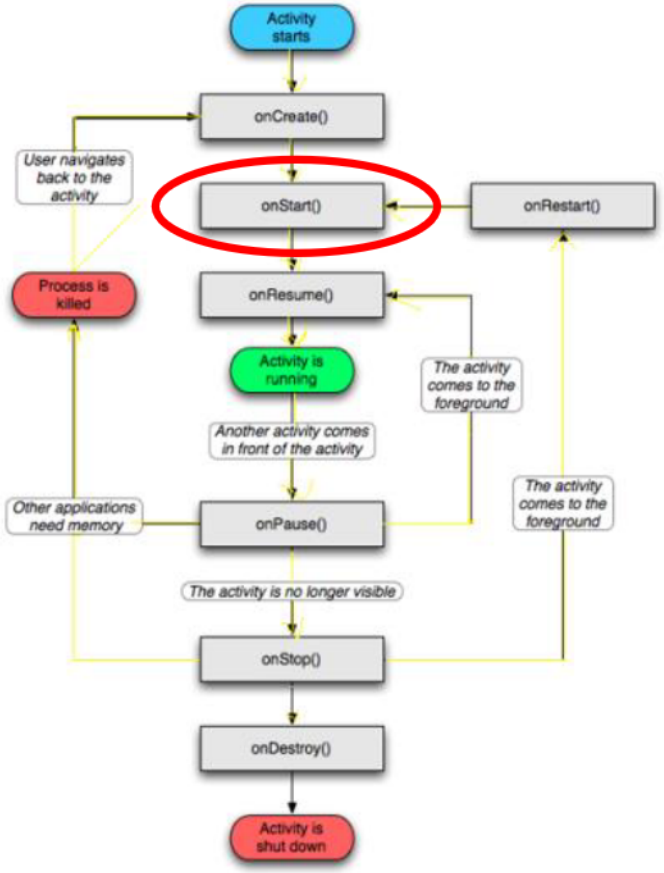
Ciclo de vida de una actividad

- Método **onCreate**():
  - **Obligatorio**: crea la interfaz de usuario (UI)
  - Parámetro **Bundle** (inicialización y/o estado)
  - Estado final: **Creada**
- Se puede emplear para rediseñar la interfaz, inicializar datos, vinculaciones, etc.
- Se puede usar para obtener los parámetros con los que se ha llamado a la actividad (getIntent())



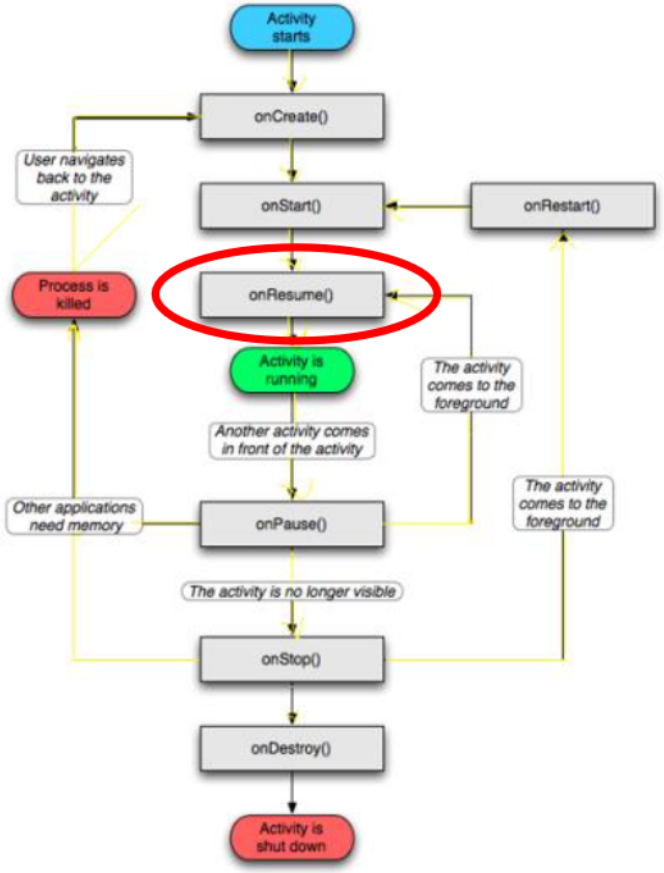
Ciclo de vida de una actividad

- Método **onStart**()
- Realiza los preparativos finales para pasar a primer plano y convertirse en interactiva
- Estado final: **Iniciada** y visible



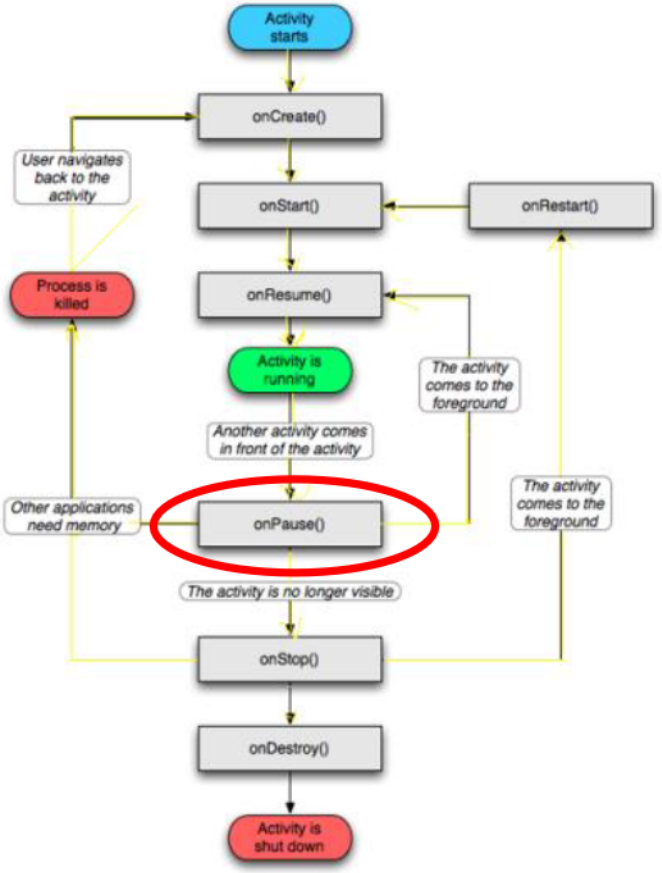
Ciclo de vida de una actividad

- Método **onResume**()
  - Se llama justo antes de obtener el foco
- Al final de la misma, la actividad:
  - Tiene el foco
  - Para a ser la primera de la pila
  - Recibe todas las interacciones del usuario
- Estado final: **Interactiva**



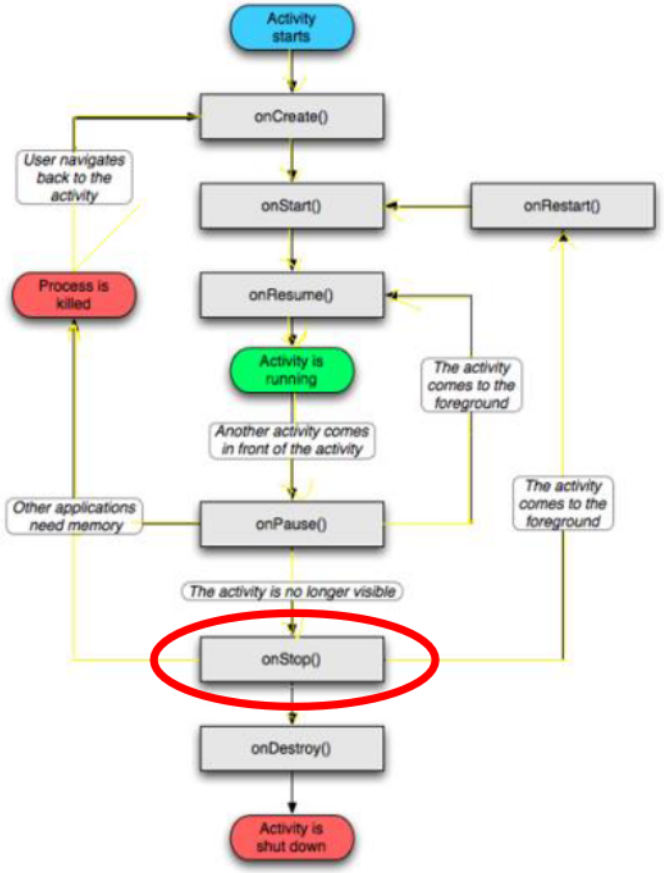
Ciclo de vida de una actividad

- Método **onPause**()
  - Se llama cuando otra actividad tiene el foco
  - p.e. usuario pulsa botón *Casa* o *Recientes*
- Estado final: **Detenida** y aún visible
  - Aún se puede actualizar (y ver) la UI
- Debe ser **muy rápido** (evitar *lags*)
  - Evitar salvar datos, transacciones o acceso a red
  - Parar procesos costosos o innecesarios



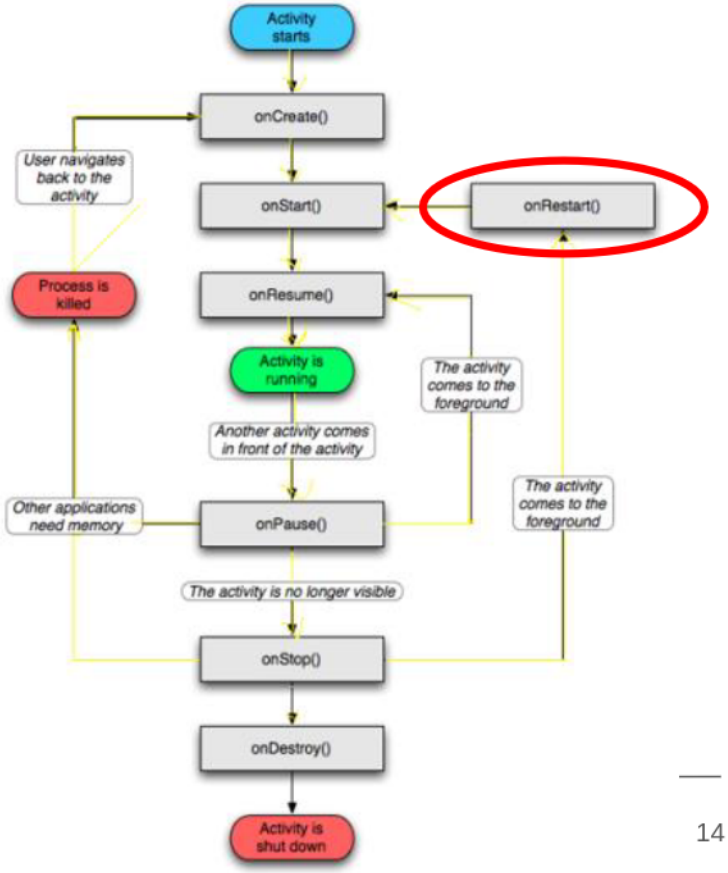
Ciclo de vida de una actividad

- Método **onStop**()
- La actividad ya no es visible para el usuario:
  - Se está eliminando
  - Se inicia una nueva
  - O una ya existente se reanuda
- Estado final: **Parada** y oculta
  - Se puede reanudar o eliminar



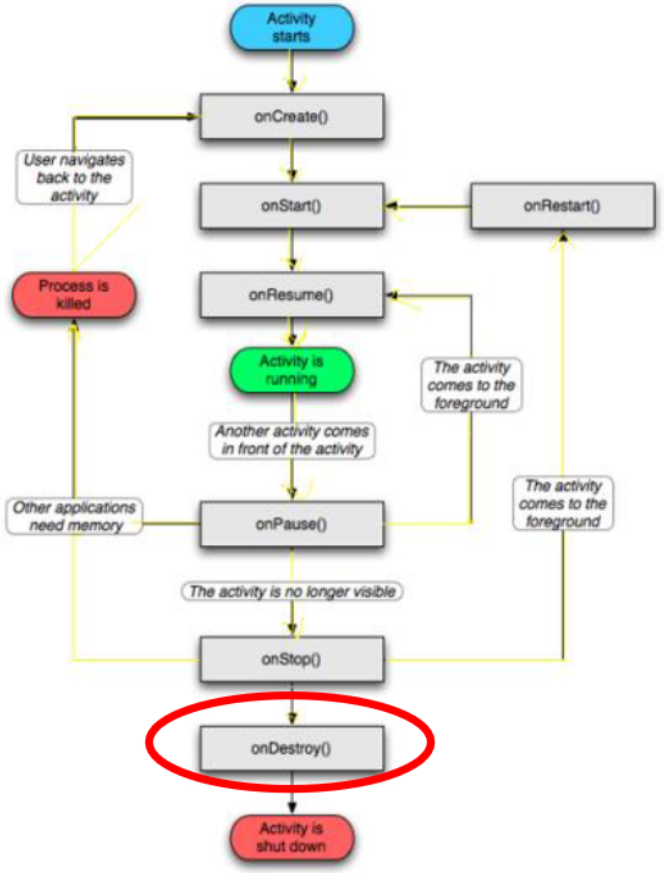
Ciclo de vida de una actividad

- Método **onRestart**()
  - Similar a onCreate()
  - Se aplica a una actividad parada
  - Después *siempre se ejecuta onStart()*
- Estado final: **Iniciada** y visible



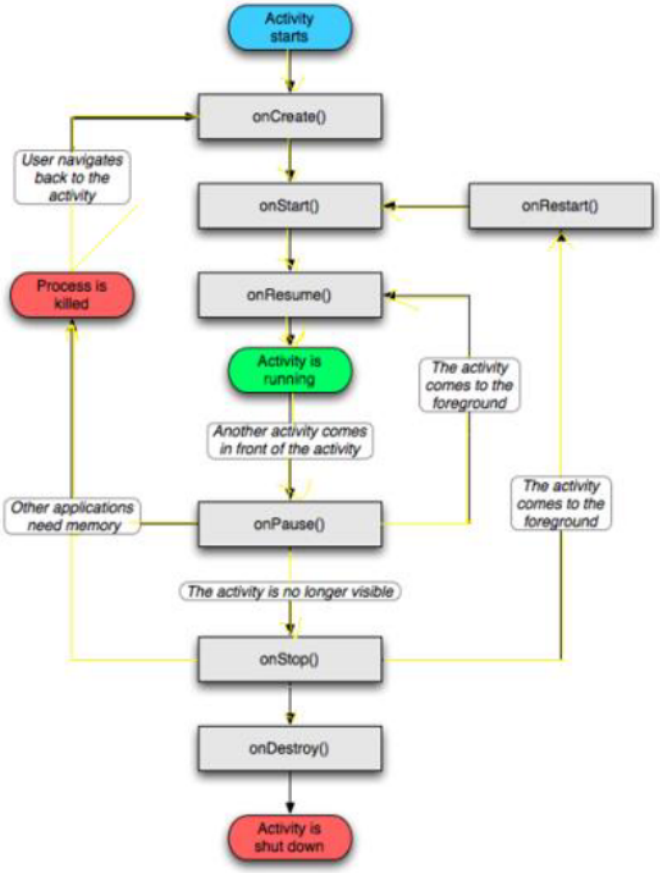
Ciclo de vida de una actividad

- Método **onDestroy**()
- Es la última antes de eliminar a la actividad
  - Botón *Atrás*
  - El sistema debe liberar recursos
  - Llamada a *finish*() (no está recomendado)
- Estado final: **Destruída**
- Se liberan todos los recursos



Ciclo de vida de una actividad

- El ciclo de vida tiene 3 lazos (“*parejas*”)
- Ciclo completo: entre onCreate() y onDestroy()
  - Recursos principales de la actividad
- Ciclo visible: entre onStart() y onStop()
  - Liberar recursos innecesarios o costosos
- Ciclo interactivo: entre onResume() y onPause()
  - Código muy ligero y rápido: evitar *lags*



Índice

1. Actividad
2. Ciclo de vida de una actividad
3. **Estado de una actividad**
4. Gestión básica de eventos en layouts
5. Recursos de una app
6. Acceso a los recursos
7. Recursos alternativos

Estado de una actividad

- El estado de la actividad está en memoria, incluso aunque no esté visible
  - Se pierde si la actividad es destruída
  - **Salvar estado**: *onSaveInstanceState(Bundle)*
  - **Recuperar estado**: *onRestoreInstanceState(Bundle), onCreate(Bundle)*
- También alguna **información adicional**: costes de serialización y deserialización
  - Mejor usar mecanismos de persistencia que ya se verán
  - OJO: No está garantizado, mejor *onPause* (atención a los costes y tiempos !!)
- Se guarda automáticamente **estado de la vista**: sólo elementos con **android:id**

Estado de una actividad

- Ejemplo: **Salvar estado** (2 variables int)

```
static final String STATE_SCORE = "playerScore";
static final String STATE_LEVEL = "playerLevel";
// ...

@Override
public void onSaveInstanceState(Bundle savedInstanceState) {
    // Save the user's current game state
    savedInstanceState.putInt(STATE_SCORE, currentScore);
    savedInstanceState.putInt(STATE_LEVEL, currentLevel);

    // Always call the superclass so it can save the view hierarchy state
    super.onSaveInstanceState(savedInstanceState);
}
```

Estado de una actividad

- Ejemplo: **Recuperar estado** (2 variables int) al inicio de la actividad
  - Comprobar si argumento Bundle es nulo (siempre lo será la primera vez)

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState); // Always call the superclass first

    // Check whether we're recreating a previously destroyed instance
    if (savedInstanceState != null) {
        // Restore value of members from saved state
        currentScore = savedInstanceState.getInt(STATE_SCORE);
        currentLevel = savedInstanceState.getInt(STATE_LEVEL);
    } else {
        // Probably initialize members with default values for a new instance
    }
    // ...
}
```

Estado de una actividad

- Ejemplo: **alternativa para recuperar estado**
  - Se llamaría después del método *onStart*
  - Sólo se llama si hay estado almacenado: No es necesario comprobar si es nulo
  - OJO: llamar siempre a la superclase para restablecer la jerarquía de las vistas

```
public void onRestoreInstanceState(Bundle savedInstanceState) {
    // Always call the superclass so it can restore the view hierarchy
    super.onRestoreInstanceState(savedInstanceState);

    // Restore state members from saved instance
    currentScore = savedInstanceState.getInt(STATE_SCORE);
    currentLevel = savedInstanceState.getInt(STATE_LEVEL);
}
```

Índice

1. Actividad
2. Ciclo de vida de una actividad
3. Estado de una actividad
4. **Gestión básica de eventos en layouts**
5. Recursos de una app
6. Acceso a los recursos
7. Recursos alternativos

Gestión básica eventos entrada

- **Objetos de escucha de eventos**: constan de dos pasos obligatorios:
  - Definir la función que se ejecuta (método de escucha o *listener*)
  - Registrar el método de escucha a elemento(s) de la vista
- Existen varios mecanismos:
  - Definición y asignación de un *listener* previo (se puede "reutilizar")
  - *Listener* anónimo (el más habitual: uno por elemento de la vista, algo pesado)
  - Anidar la interfaz en la propia actividad (más ligero, se chequea en compilación, recuerda los métodos necesarios, pero es el mismo para todos !!)

Gestión básica eventos entrada

- **Algunos ejemplos** de interfaces de los objetos de escucha y cuando se llaman:
  - *onClick*(), View.OnClickListener: el usuario toca el elemento
  - *onLongClick*(), View.OnLongClickListener: el usuario presiona elemento (1 seg)
  - *onFocusChange*(), View.OnFocusChangeListener: el usuario navega hacia el elemento o sale de él
  - *onCreateContextMenu*(), View.OnCreateContextMenuListener: se crea un menú contextual (resultado de un “clic largo” sostenido)
- Cada **interfaz tiene su propia función de registro** (ver la documentación)

Estado de una actividad

- Ejemplo definiendo un listener:

```
// Create an anonymous implementation of OnClickListener
private OnClickListener corkyListener = new OnClickListener() {
    public void onClick(View v) {
      // do something when the button is clicked
    }
};

protected void onCreate(Bundle savedValues) {
    ...
    // Capture our button from layout
    Button button = findViewById(R.id.corky);
    // Register the onClick listener with the implementation above
    button.setOnClickListener(corkyListener);
    ...
}
```

Estado de una actividad

- Ejemplo gestión eventos botón:

```
public class ExampleActivity extends Activity implements OnClickListener {
    protected void onCreate(Bundle savedValues) {
        ...
        Button button = findViewById(R.id.corky);
        button.setOnClickListener(this);
    }

    // Implement the OnClickListener callback
    public void onClick(View v) {
      // do something when the button is clicked
    }
    ...
}
```

Gestión básica eventos entrada

- **Controladores de eventos**:
  - Son genéricos (se usan poco porque no están asociados a elementos de la vista)
  - Se suelen usar para personalizar o crear nuevos componentes de la vista
- Controladores más usados y cuando se llama o activa cada uno:
  - *onKeyDown(int, KeyEvent)*: evento de tecla nuevo
  - *onKeyUp(int, KeyEvent)*: evento de activación de tecla
  - *onTrackballEvent(MotionEvent)*: evento de movimiento (bola de seguimiento)
  - *onTouchEvent(MotionEvent)*: evento de movimiento de la pantalla táctil

Referencias elementos de un layout

- Método antiguo: **findViewById()** p.e. *Button but = findViewById(R.id.but)*
  - Poco fiable: no se comprueban las referencias durante la programación
- Nuevo método: **view bindings** seguro y simple de usar, cuesta algo más configurar
  1. En el fichero *build.grade* del módulo añadir en *android* las líneas (y luego *sync*)

     buildFeatures { viewBinding = true }
  2. En el onCreate de cada actividad crear la variable `ActivityMainBinding binding;`
     - el nombre de esta clase deriva del layout correspondiente: *activity_main.xml*
  3. Sustituir `setContentView(R.layout.activity_main);` por:

     ```
     binding = ActivityMainBinding.inflate(getLayoutInflater());
     View view = binding.getRoot();
     setContentView(view);
     ```
  4. El sistema solo permite usar los elementos de ese layout: p.e. `binding.butMain`

Índice

1. Actividad
2. Ciclo de vida de una actividad
3. Estado de una actividad
4. Gestión básica de eventos en layouts
5. **Recursos de una app**
6. Acceso a los recursos
7. Recursos alternativos

Recursos app

- Una app está compuesta de código y recursos
  - Los recursos son los **ficheros adicionales y el contenido estático de la app**
- Las motivaciones para usar recursos son:
  - Mantener los datos de la app de forma independendiente (externalizar)
  - Proporcionar opciones para varias configuraciones (idioma, pantalla, teclado, ...)
  - Minimizar dependencias con el código y recompilaciones innecesarias
  - Facilita cambios y adaptaciones automáticos en tiempo de ejecución

Recursos app

- Se declaran con etiquetas XML en **/res**:
  - Se agrupan en subdirectorios y ficheros
  - Todos se identifican con la etiqueta ***<resource>***
  - Android crea una única clase R con todos ellos (!!)
- En la ejecución se selecciona el conjunto apropiado de recursos según la configuración del dispositivo:
  - **Adaptación automática** sin necesidad de recompilar
- Cada recurso tiene obligatoriamente:
  - **Nombre**: debe ser único en toda la app/paquete
  - **Tipo** (ver documentación para conocerlos todos)

```
MyProject/
    src/
        MyActivity.java
    res/
        drawable/
            graphic.png
        layout/
            main.xml
            info.xml
        mipmap/
            icon.png
        values/
            strings.xml
```

Recursos app: tipos básicos

● **res/animator/**: animación propiedades	● R.animator.	● <animator>
● **res/anim/**: animación por interpolación	● R.anim.	● <anim>
● **res/color/**: definición de colores	● R.color.	● <color>
● **res/drawable/**: gráficos y mapas de bits	● R.drawable.	● <drawable>
● **res/layout/**: diseño interfaces usuario	● R.layout.	● <layout>
● **res/mimap/**: elementos diseño e iconos	● R.mimap.	● <mimap>
● **res/menu/**: diseño de todos los menús	● R.menu.	● <menu>
● **res/raw/**: elementos no procesables	● R.raw.	● <raw>
● **res/values/**: nombres, estilos, enteros, colores, dimensiones, arrays, etc	● R.string., R.integer., R.style., R.array.	● <string>, <integer>, <integer>, <array>
● **res/xml/**: ficheros XML (configuración)	● R.xml.	● <xml>

Recursos app: gestión

- En ciertas situaciones puede resultar desaconsejable reiniciar la actividad
  - Retener la información relevante en un objeto
    - Es decir, independizar la vista/actividad de los datos
  - **Gestionar internamente el cambio de configuración** (poco recomendado)
    - Uso de android:configChanges en el manifiesto
  - ***Forzar* la configuración de la actividad** en el manifiesto:
    - Ejemplo fijar orientación pantalla: *android:screenOrientation="landscape"*
      - Existen otros valores (ver documentación): *portrait, user, sensor*, ...

Índice

1. Actividad
2. Ciclo de vida de una actividad
3. Estado de una actividad
4. Gestión básica de eventos en layouts
5. Recursos de una app
6. **Acceso a los recursos**
7. Recursos alternativos

Acceso a los recursos app

- Un recurso se referencia mediante un código o etiqueta XML **único**:
  - Paquete: no se necesita indicar si la referencia es local
  - Tipo: está entre los ya comentados (hay más en la documentación oficial)
  - Nombre: se corresponde con el atributo *android:name* en la definición

- Referencias **XML**:

@[<*package_name*>:]<*resource_type*>/<*resource_name*>

- Referencias **Java**:

[<package_name>.]R.<resource_type>.<resource_name>

Acceso a los recursos app

- Ejemplo con varias referencias XML:

values.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
            <color name="opaque_red">#f00</color>
            <string name=”labelButton”> Submit </string>
            <string name=”labelText”> Hello world! </string>
</resources>
```

main_activ.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
            <Textview android:id=“@+id/label1” android:text=“@string/labelText”
            android:textcolor=“@color/opaque_red” >    </Textview>
            <Button android:id=“@+id/button1” android:text=“@string/labelButton”> </Button>
</resources>
```

Acceso a los recursos app

- Ejemplo con varias referencias Java

main_activity.java

```
// Get a string resource from the string.xml file
String hello = getResources().getString(R.string.hello);

// Get a color resource from the string.xml file
final int color = getResources().getColor(R.color.opaque_red);

// Load a custom layout for the current screen
setContentView(R.layout.main_activ);

// Set the text on a TextView object using a resource ID
TextView msgTextView = findViewById(R.id.label1);
msgTextView.setText(R.string.labelText);
```

Acceso a los recursos app

- También existen **recursos del sistema** accesibles a todas las apps
  - Mecanismo “simple” para obtener iconos, layout, etc
  - Válidos entre diferentes versiones de Android y fabricantes
- Se identifican con el nombre de paquete **android**:
  - **XML**: @android:R/tipo/nombre
  - **Java**: android.R.<tipo>.<nombre>

Acceso a los recursos app

- Acceso a los recursos originales (sólo de lectura) y datos sin procesar:
  - Directorio **res/raw/**:
    - Acceso con método *openRawResource()* a través del ID asignado
    - Lectura a través del flujo de bytes (gestionar!!)
    - No puede haber subdirectorios
  - Directorio **assets/**:
    - No tienen ID asignado
    - Acceso similar a un archivo normal y usando *AssetManager*
    - Se pueden crear subdirectorios anidados

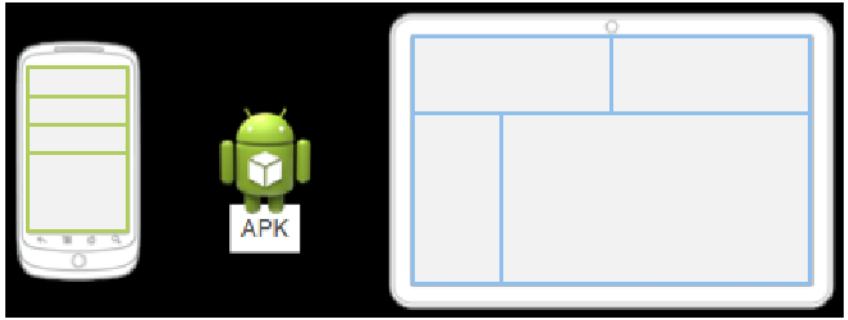
Índice

1. Actividad
2. Ciclo de vida de una actividad
3. Estado de una actividad
4. Gestión básica de eventos en layouts
5. Recursos de una app
6. Acceso a los recursos
7. **Recursos alternativos**

Recursos alternativos

- Para especificar alternativas crear:
  - ***res/<resource_name>_<config_qualifier>***
  - Puede haber más de un calificador a la vez
  - Los directorios no pueden estar anidados
  - No distingue mayúsculas y minúsculas
- Ejemplo de 2 configuraciones:
  - Diferentes tamaños de pantalla

```
res/
    drawable/
        icon.png
        background.png
    drawable-hdpi/
        icon.png
        background.png
```

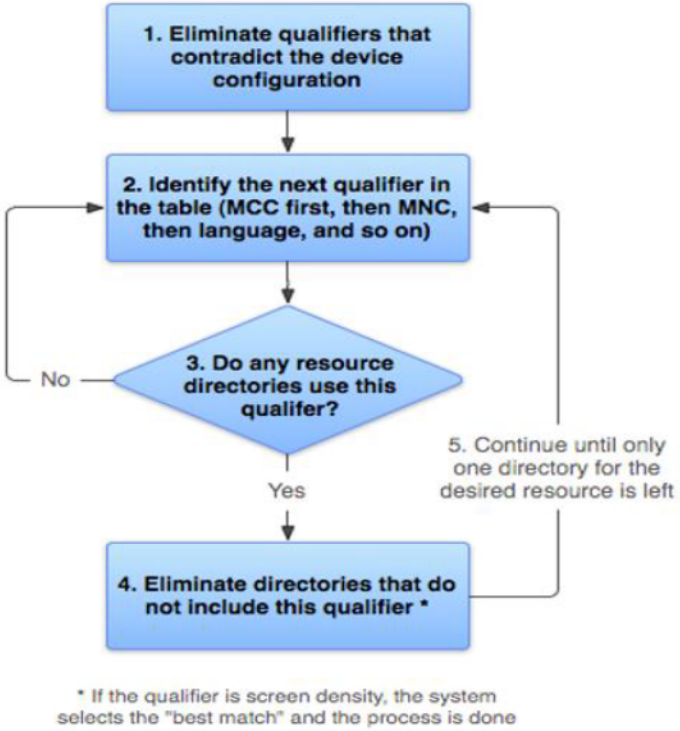


Recursos alternativos: Qualifiers

Configuration	Values Example	Description
1.MCC and MNC	mcc310, mcc208, etc	mobile country code (MCC)
2.Language and region	en, fr, en-rUS, etc	ISO 639-1 language code
3.smallestWidth	sw320dp, etc	shortest dimension of screen
4.Available width	w720dp, w320dp, etc	minimum available screen width
5.Available height	h720dp, etc	minimum available screen height
6.Screen size	small, normal, large	screen size expressed in dp
7.Screen aspect	long, notlong	aspect ratio of the screen
8.Screen orientation	port, land	screen orientation (can change!)
9.Screen pixel density (dpi)	ldpi, mdpi, hdpi	screen pixel density
10.Keyboard availability	keysexposed, etc	type of keyword
11.Primary text input method	nokeys, qwerty	availability of qwerty keyboard
12.Navigation key availability	navexposed, etc	navigation keys of the application
13.Platform Version (API)	v3, v4, v7, etc	API supported by the device

Recursos alternativos

- Selección de recursos alternativos:
  - Determinar configuración del dispositivo en tiempo de ejecución
  - Seleccionar los recursos según prioridad
- Siempre debe existir **una configuración por defecto** (sin calificador)



Recursos alternativos

- Supongamos la siguiente configuración de recursos alternativos

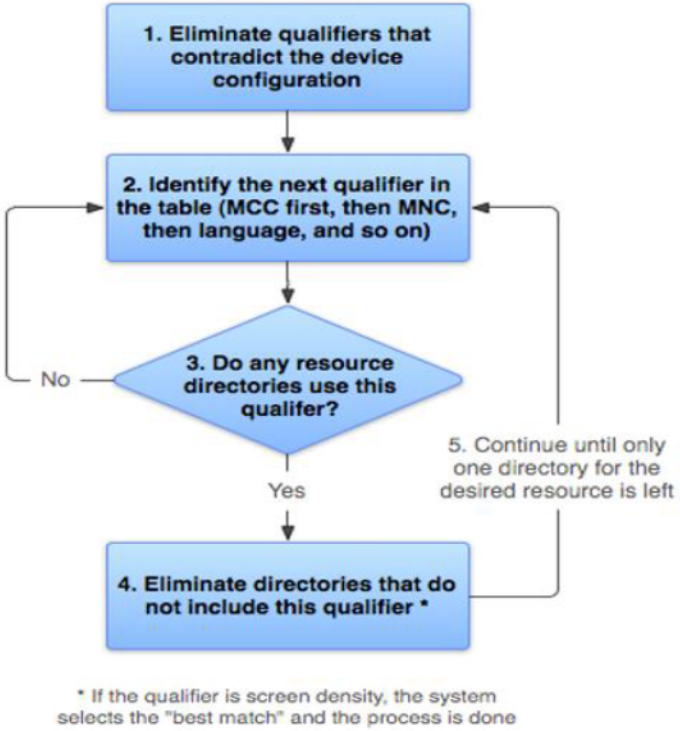
```
drawable/
drawable-en/
drawable-fr-rCA/
drawable-en-port/
drawable-en-notouch-12key/
drawable-port-ldpi/
drawable-port-notouch-12key/
```

- Supongamos el siguiente dispositivo:
  - Configuración regional = en-GB
  - Orientación de la pantalla = port
  - Densidad de píxeles de la pantalla = hdpi
  - Tipo de pantalla táctil = notouch
  - Método principal de entrada de texto = 12key

Recursos alternativos

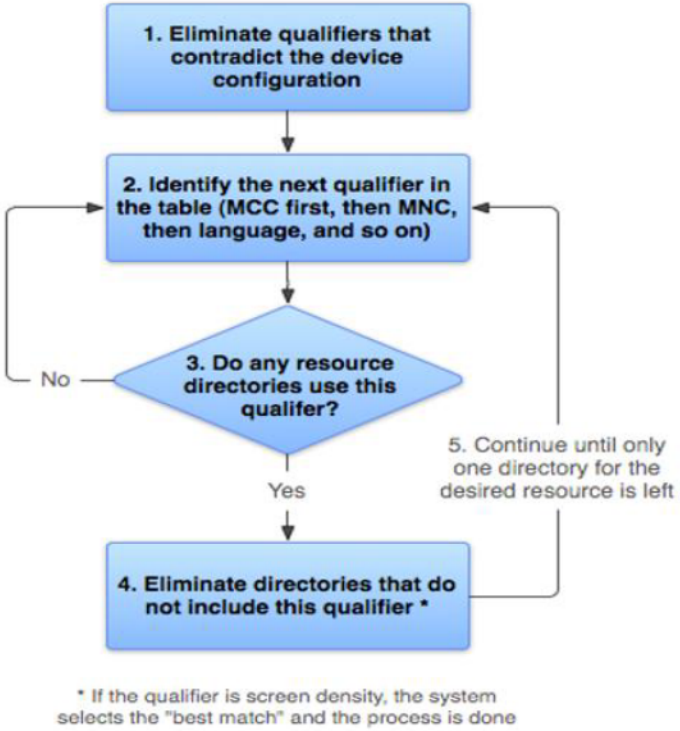
1. Se eliminan los recursos en conflicto con la configuración del dispositivo:
   a. *drawable-fr-rCA* contradice *en-GB*

```
drawable/
drawable-en/
drawable-fr-rCA/
drawable-en-port/
drawable-en-notouch-12key/
drawable-port-ldpi/
drawable-port-notouch-12key/
```



Recursos alternativos

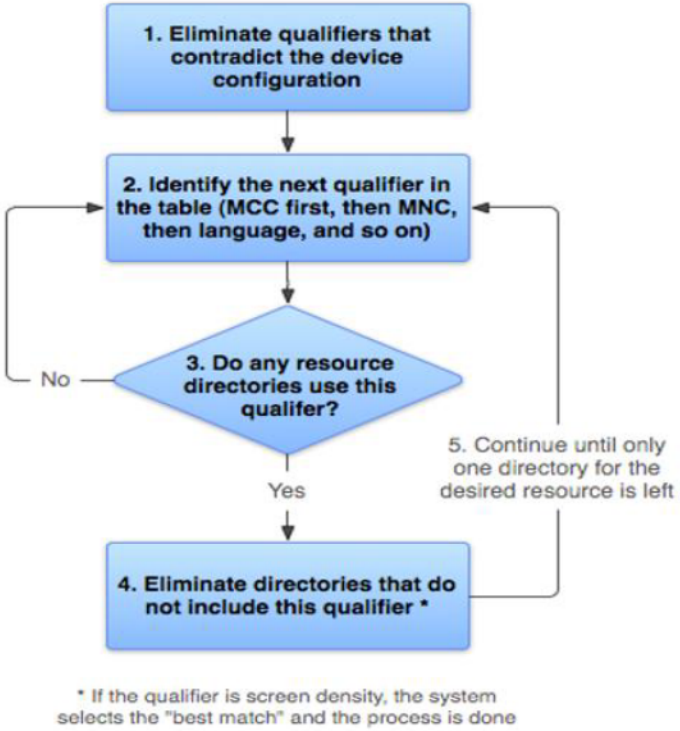
2. Se selecciona el (siguiente) calificador:
    a. En la tabla previa están definidos todos los calificadores de mayor a menor prioridad
3. Se comprueba si el calificador está definido en los recursos alternativos de la app:
    a. No: se pasa al siguiente calificador (paso 2)
    b. Si: se pasa al paso 4



Recursos alternativos

4. Se eliminan los directorios que no incluyan el calificador seleccionado:
   a. En el ejemplo todos los que no incluyen un idioma

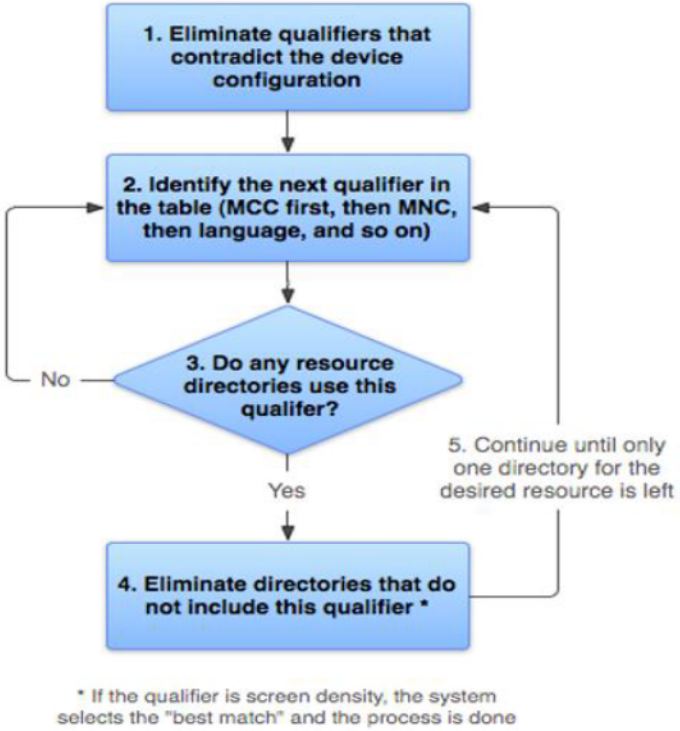
```
~~drawable/~~
drawable-en/
drawable-en-port/
drawable-en-notouch-12key/
~~drawable-port-ldpi/~~
~~drawable-port-notouch-12key/~~
```



Recursos alternativos

5. Repetir pasos 2, 3 y 4 hasta quedar 1 directorio:
   a. En el ejemplo aquellos que no incluyan orientación de pantalla
   b. Queda el directorio: *drawable-en-port*

```
~~drawable-en/~~
drawable-en-port/
~~drawable-en-notouch-12key/~~
```



- Predomina prioridad al número de calificadores:
  - *drawable-en* vs. *drawable-port-notouch-12key*

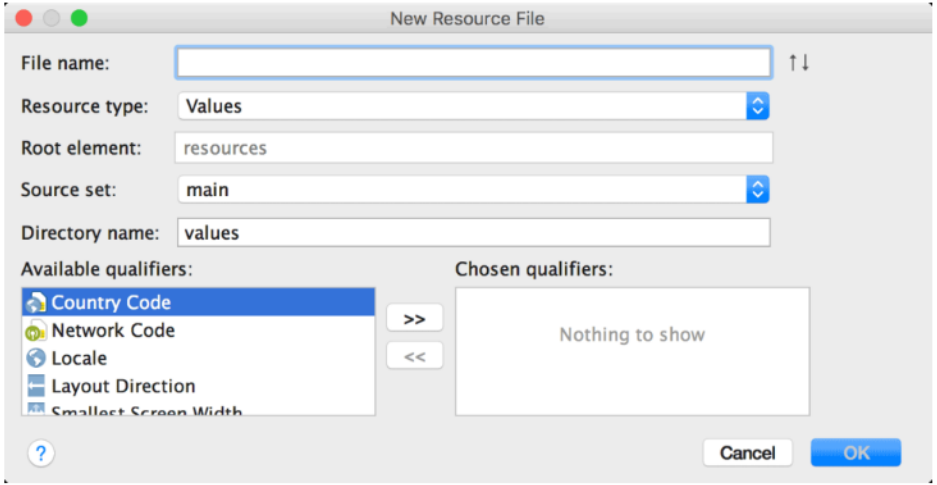
Recursos alternativos: ejemplos

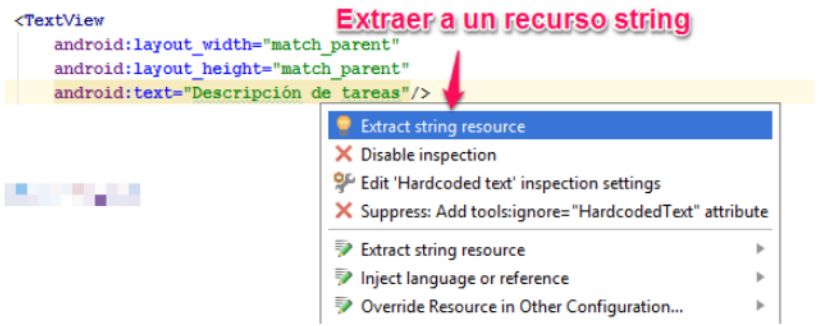
- Varios idiomas:
  - *res/values* (idioma por defecto)
  - *res/values-en* (Inglés)
  - *res/values-fr* (Francés)
  - *res/values-de* (Alemán)
- Se pueden añadir una o varias adaptaciones regionales
- No es obligatorio tener todas las traducciones

- Varias orientaciones de pantalla:
  - *res/layout* (por defecto)
  - *res/layout-port* (retrato) o
  - *res/layout-land* (apaisado)
- Sólo una opción adicional

Recursos alternativos: Android Studio

- Android Studio tiene un **asistente** para facilitar operaciones con recursos
  - En ventana *Project* (*Android o Project*) pulsar *File > New > Android resource file*
  - También pulsando botón derecho en directorio *res/* o elemento de interés





Recursos alternativos: Android Studio



Referencias

- https://developer.android.com/reference/android/app/Activity
- https://developer.android.com/guide/components/activities
- https://developer.android.com/guide/components/activities/activity-lifecycle
- https://developer.android.com/guide/topics/ui/ui-events
- https://developer.android.com/guide/topics/resources
- https://developer.android.com/studio/write/add-resources
- https://developer.android.com/topic/libraries/view-binding